



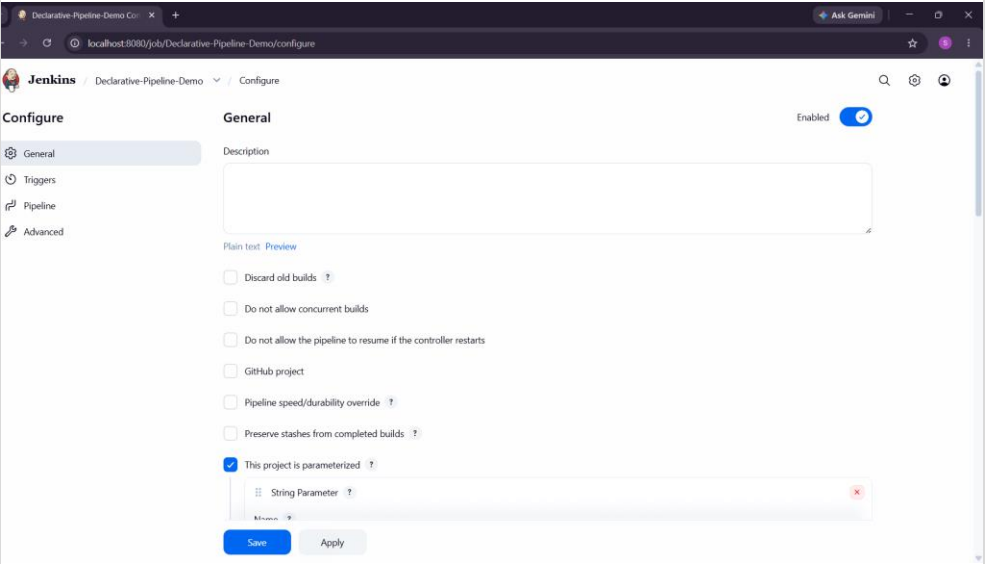
SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 6	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Basic Declarative Pipeline
Theory	Theory Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates software development tasks such as building, testing, and deploying applications. Jenkins integrates with version control systems like Git to automatically retrieve source code and execute predefined build steps. In this experiment, Jenkins was installed using Docker, configured with the Git plugin, and connected to a GitHub repository. A Freestyle project was created to pull source code from the repository and execute shell commands. The build was then triggered manually, and the console output was examined to verify successful execution. Commands Used • docker --version – Verified that Docker was installed on the system. • docker ps – Checked whether the Jenkins Docker container was running. • docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name myjenkins jenkins/jenkins:lts-jdk17 – Created and started the Jenkins Docker container. • docker logs myjenkins – Displayed the Jenkins logs to obtain the initial administrator password during the initial setup. • docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword – Retrieved the initial administrator password generated by Jenkins. Execute Shell Commands • echo "BUILDING: \${JOB_NAME}" – Displayed the Jenkins job name. • echo "BUILD NUMBER: \${BUILD_NUMBER}" – Displayed the current build number. • echo "WORKSPACE: \${WORKSPACE}" – Displayed the Jenkins workspace path.

	• <code>ls -la</code> – Listed all files present in the Jenkins workspace. <code>echo "Hello Jenkins!"</code> – Displayed a sample message to verify successful execution of the build step.
Implementation	Step 1: Create a New Pipeline Project  Verified Docker and Jenkins Container — Verified Docker installation and confirmed that the Jenkins Docker container (myjenkins) was running successfully. Step 2: Define the Pipeline Script <pre>pipeline { agent any parameters { string(name: 'GREETING_NAME', defaultValue: 'World', description: 'The name to use in the greeting') } stages { stage('Greeting') { steps { echo 'Running Windows Pipeline...' bat "echo Hello \${params.GREETING_NAME}!" } } stage('Checkout Source Code') { steps { echo 'Starting Git checkout...' git url: 'https://github.com/Kranvagn4/Git_prac.git', branch: 'main' echo 'Git checkout completed.' } } stage('Run Example Command') { steps { echo 'Executing Windows commands...' bat 'echo --- Reading README ---' bat 'type README.md' } } } }</pre>

```

    }

    post {
        always {
            echo 'Pipeline finished.'
        }
    }
}
}

```

Defined Declarative Pipeline Script — Configured the Jenkins Pipeline using a Declarative Pipeline script to check out source code from the GitHub repository and execute shell commands in separate stages.

Step 3: Save and Run the Pipeline

 #2 (Jul 23, 2026, 11:37:14 AM)



Started by user [Sanyukt Ramola](#)



Revision: 9c15df12174d7d5aff998acfb81b2792fee601a

Repository: https://github.com/Kranvagn4/Git_prac.git

- refs/remotes/origin/main



No changes.

Executed Declarative Pipeline — Successfully executed the Declarative Pipeline and verified that all stages, including Checkout Source Code and Run Example Command, completed successfully.

Step 4: View Console Output

```

Started by user Sanyukt Ramola
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Greeting)
[Pipeline] echo
Running Windows Pipeline...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo>echo Hello World!
Hello World!
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Checkout Source Code)
[Pipeline] echo
Starting Git checkout...
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/Kranvagn4/Git_prac.git # timeout=10
Fetching upstream changes from https://github.com/Kranvagn4/Git_prac.git
> git.exe --version # timeout=10
> git --version # 'git version 2.46.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/Kranvagn4/Git_prac.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 9c15df12174d7d5aff998acfb81b2792fee601a (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 9c15df12174d7d5aff998acfb81b2792fee601a # timeout=10
> git.exe branch -a -v --no-abbrev # timeout=10

```

```
> git.exe branch -D main # timeout=10
> git.exe checkout -b main 9c15df12174d7d5aff998acfb81b2792fee601a # timeout=10
Commit message: "Update README.md"
> git.exe rev-list --no-walk 9c15df12174d7d5aff998acfb81b2792fee601a # timeout=10
[Pipeline] echo
Git checkout completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Example Command)
[Pipeline] echo
Executing Windows commands...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo>echo --- Reading README ---
--- Reading README ---
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo>type README.md
# Awesome Git Practice Project (Deature Update)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Verified Console Output — Examined the console output and verified that Jenkins successfully checked out the Git repository, executed the pipeline stages, ran the shell commands, and completed the pipeline with a SUCCESS status.

Subject Faculty Name: Prof. Arijit Dutta